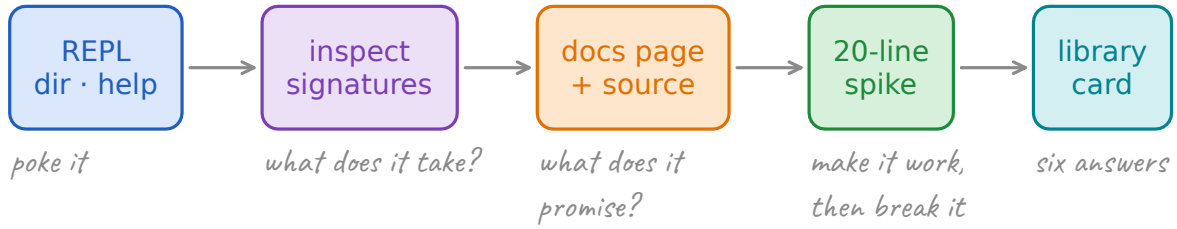
Chapter 7

How to learn any library

the method that makes every other chapter optional reading

When `sqlite3` or `hmac` feels impossible, the problem is almost never syntax. You can call a function and write a `with` block. What's missing is the **shape** of the library: what you create first, what you call on it, what you must close, and what goes wrong.

This chapter is the method for finding that shape on your own, for any module, in any language. It has four tools and one output:



Every later chapter is this method already applied to one module. Once it's a habit, you won't need a chapter (or a tutorial) to start using an unfamiliar library.

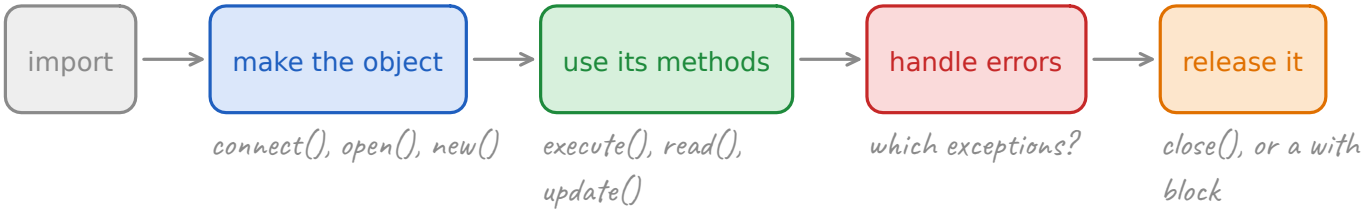
> **KEY IDEA**
> You don't learn a library by reading all of it. You learn its **shape** (the six card questions), then look up details only when you need them.

The four shapes a library comes in

Nearly every library is one of four shapes. Spotting which one tells you how to start:

shape	how you use it	examples
Bag of functions	import, call functions, get values back. Nothing to close.	`math`, `json`, `base64`, `statistics`, `difflib`
Object factory	a function makes an object; you call its methods; often you must close it.	`sqlite3.connect`, `open`, `hmac.new`, `zipfile.ZipFile`
Class to subclass	you write a subclass and fill in methods; the library calls them.	`unittest.TestCase`, `http.server`'s `BaseHTTPRequestHandler`
Framework / runtime	you hand over your functions; the library runs the loop.	`asyncio.run`, `tkinter` mainloop, `argparse` (lighter)

The object factory shape is where most people get stuck, because the important part is the **lifecycle**: the order of calls, and what must happen at the end.




```python
import zipfile, io

buf = io.BytesIO()                          # an in-memory 'file'
with zipfile.ZipFile(buf, "w") as z:        # make the object
    z.writestr("hello.txt", "hi there")     # use its methods
# leaving the with block released it (wrote the zip's index)

with zipfile.ZipFile(buf) as z:
    print(z.namelist(), z.read("hello.txt"))
```

```
['hello.txt'] b'hi there'
```

TRAP

Forgetting the release step causes the bugs that make no sense: a zip with no index, a database change that never saved, a file that's still empty. If a library hands you an object, your first question is **how do I close it?**

Tool 1: the REPL, with dir() and help()

Open `python3` and import the module. `dir()` lists every name on an object; filter out the underscore names and what's left is the **public surface**:

```
>>> import hmac
>>> [n for n in dir(hmac) if not n.startswith("_")]
['HMAC', 'compare_digest', 'digest', 'digest_size', 'new', 'trans_36', 'trans_5C']
>>> type(hmac.new), type(hmac.HMAC)
(<class 'function'>, <class 'type'>)
```

Now you know: one function (`new`), one class (`HMAC`), one helper (`compare_digest`). The `trans_` names are internal tables a stricter author would have hidden. `help(name)` shows the docstring; `__doc__` gives the same text as a string, handy for the first lines:

```
>>> print(hmac.new.__doc__[:200])
Create a new hashing object and return it.

key: bytes or buffer, The starting key for the hash.
msg: bytes or buffer, Initial input for the hash, or None.
digestmod: A hash name suitable for hashlib.
```

The factory makes an object. Make one, and `dir()` **that** too: the object's methods are the next layer of the shape.

```
>>> h = hmac.new(b"secret key", b"message", "sha256")
>>> print(*[n for n in dir(h) if not n.startswith("_")])
block_size blocksize copy digest digest_size hexdigest name update
>>> h.hexdigest()[:16]
'c431e4857a6b817f'
```

in the REPL	tells you
`dir(x)`	every attribute name (filter the underscores)
`help(x)` / `x.__doc__`	the docstring: what it does and what it takes
`type(x)`, `type(x).__mro__`	what kind of object, and which classes it inherits from
`x.__module__`, `module.__file__`	where it lives, so you can read the source
`callable(x)`, `isinstance(x, type)`	function or class? (filtering `dir()` output)

Tool 2: inspect, signatures and the source

`inspect.signature` answers "what does it take?" exactly. Learn to read the two markers, because the docs use them too:

```
>>> import inspect, json
>>> inspect.signature(sorted)
<Signature (iterable, /, *, key=None, reverse=False)>
>>> inspect.signature(hmac.new)
<Signature (key, msg=None, digestmod='')>
>>> inspect.signature(json.dumps)
<Signature (obj, *, skipkeys=False, ensure_ascii=True, check_circular=True, allow_nan…
```

in a signature	means	so you write
`a, b`	ordinary: by position or by name	`f(1, 2)` or `f(a=1, b=2)`
`x, /`	everything before `/` is **positional-only**	`sorted(xs)`, never `sorted(iterable=xs)`
`*, key=None`	everything after `*` is **keyword-only**	`sorted(xs, key=len)`, not `sorted(xs, len)`
`msg=None`	has a default, so optional	leave it out, or pass it
`*args`, `**kw`	any extra positional / keyword arguments	read the docs for what's accepted
`[, x]` in docs	older docs' notation for optional	same as a default

Then read the source. A surprising amount of the standard library is plain Python, and reading it is the fastest way to see what a function really does:

```
1 import inspect, statistics
2 src = inspect.getsource(statistics.median)
3 body = src.split('"""')[2]          # skip the docstring
4 print(body.strip())
```

← the real code

↖ drop the docstring

```
data = sorted(data)
    n = len(data)
    if n == 0:
        raise StatisticsError("no median for empty data")
    if n % 2 == 1:
        return data[n // 2]
    else:
        i = n // 2
        return (data[i - 1] + data[i]) / 2
```

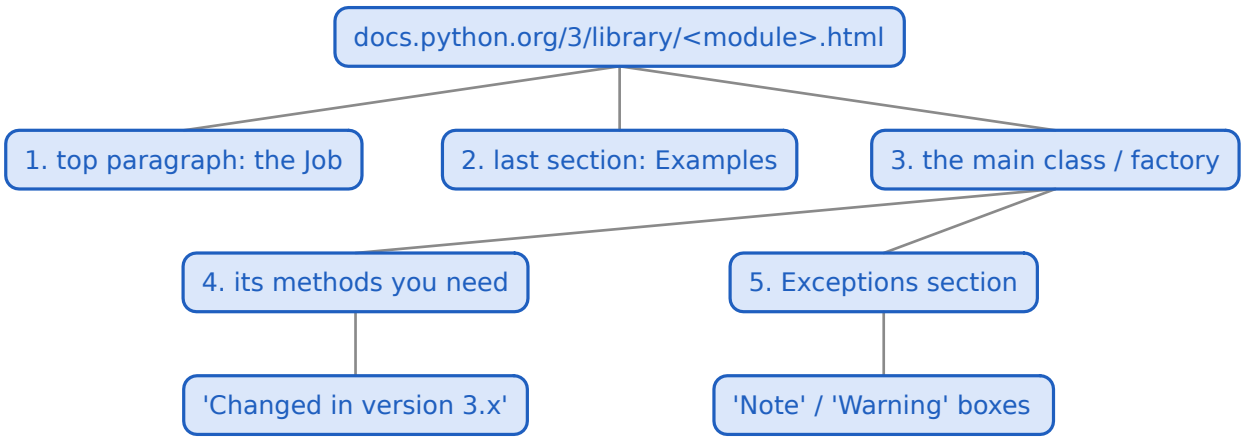
HONEST NOTE

Some modules are written in C (`math`, `_sqlite3`, parts of `hashlib`). For those `inspect.getsource` raises `TypeError`, and `inspect.signature(sqlite3.connect)` even raises `ValueError` on 3.14. Then fall back to `help()` and the docs page. `module.__file__` tells you which it is: a `.py` path, or a `.so` (compiled C).

```
>>> import math, sqlite3
>>> math.__file__.rsplit("/", 1)[-1]
'math.cpython-314-x86_64-linux-gnu.so'
>>> statistics.__file__.rsplit("/", 1)[-1]
'statistics.py'
```

▢ Tool 3: reading a docs page (without reading all of it)

A page on docs.python.org is a reference, not a story. You don't read it top to bottom. You read it in this order, and stop as soon as you can write your spike:



Skip ahead to the examples. Most module pages end with a worked example; it shows the lifecycle in ten lines. Warning boxes are where the traps live (for `hmac`: compare with `compare_digest`, not `==`). "Changed in version" lines tell you whether a blog post from 2016 still applies.

You can read the same docstrings offline in a terminal, which helps when you're on a train:

```
$ python3 -m pydoc hmac            # the module's docs, in a pager
$ python3 -m pydoc hmac.new        # one function
$ python3 -m pydoc -k zip          # search every module's summary line
```

RULE OF THUMB

Search in this order: the official docs page, then the source, then the PEP that introduced the feature (linked from the docs). Only then blogs and Q&A sites, and check the **date**: Python 2 answers still rank highly. AI answers are a fourth opinion: run what they give you in a spike before you trust it.

Tool 4: the 20-line spike

A **spike** is the smallest throwaway program that uses the library end to end. You write it in a scratch file before touching your real project. Then you **break it on purpose**, because seeing the errors now means you'll recognise them later.

Here's a spike for `difflib`, a module this book never teaches. Ten minutes with `dir()` and the docs page found `get_close_matches`:

```python
import difflib

commands = ["status", "commit", "checkout", "branch", "merge"]
for typo in ["comit", "chekout", "brnch", "xyz"]:
    print(typo, "->", difflib.get_close_matches(typo, commands, n=1))

# break it: what does the cutoff do?
print(difflib.get_close_matches("xyz", commands, cutoff=0.0))
```

← *make it work*

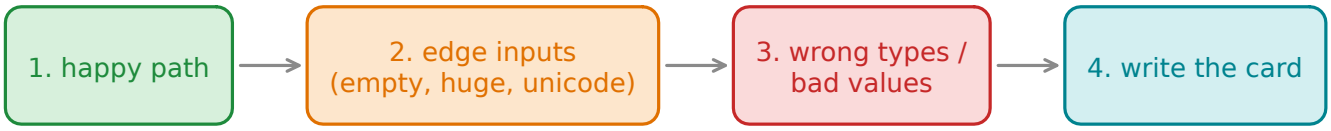
← *now poke it*

```
comit -> ['commit']
chekout -> ['checkout']
brnch -> ['branch']
xyz -> []
['status', 'merge', 'commit']
```

And break it with wrong input, to learn the error before it surprises you:

```python
difflib.get_close_matches("x", commands, n=0)
```

```
ValueError: n must be > 0: 0
```



✎ YOUR GO (pencil, or the REPL, first)

Spike the textwrap module: find (with dir and help, no searching) how to wrap a long sentence to 30 characters and how to indent every line by two spaces. Then pass it an empty string.

Worked example: cracking hmac from zero

Pretend you've never seen `hmac`. The project needs to check that a QR code's contents weren't edited. Five minutes of the method:

step	what you did	what you learned
Job	first line of `hmac.__doc__`, the top paragraph of the docs	makes a keyed hash: a tag only someone with the key could produce
Shape	`dir(hmac)`: `new`, `HMAC`, `compare_digest`, `digest`	object factory, plus a one-shot function `digest`
Signature	`inspect.signature(hmac.new)`	`(key, msg=None, digestmod='')`: key and message are bytes; you must choose a hash
Docs warning	the Warning box on the page	compare tags with `compare_digest`, never `==`
Spike	the 12 lines below	it works; wrong key gives a different tag; `str` key is a `TypeError`

```python
import hmac

KEY = b"server-only secret"
def tag(msg: bytes) -> str:
    return hmac.new(KEY, msg, "sha256").hexdigest()

t = tag(b"student=42;room=B12")
print(t[:20], len(t))
print(hmac.compare_digest(t, tag(b"student=42;room=B12")))
print(hmac.compare_digest(t, tag(b"student=43;room=B12")))
print(hmac.digest(KEY, b"student=42;room=B12", "sha256").hex() == t)
```

```
1b34cbb6d11eb6641380 64
True
False
True
```

← factory + method in one line

← unchanged: True

← edited: False

← the one-shot form agrees

```python
hmac.new("not bytes", b"m", "sha256")
```

```
TypeError: key: expected bytes or bytearray, but got 'str'
```

The output of the method is the card. Writing it down is what makes it stick, and it's what you'll look at in six months instead of the docs:

LIBRARY CARD	`hmac`
Job	A keyed hash (a **tag**): proves a message came from someone with the key and wasn't changed.
Core objects	`hmac.new(key, msg, digestmod)` → `HMAC` object with `.update(b)`, `.digest()`, `.hexdigest()`. One-shot: `hmac.digest(key, msg, 'sha256')`.
Lifecycle	new → update (any number of times) → digest. Nothing to close.
Errors	`TypeError` if key/msg are `str` (encode them); unknown digest name → `ValueError`.
Traps	Compare with `hmac.compare_digest` (**== leaks timing**). Not encryption: the message is still readable. Keep the key out of the source (Ch 13).
Docs	docs.python.org/3/library/hmac.html · RFC 2104

When you're stuck: a checklist

- **Which shape is it?** Bag of functions, factory, subclass, or framework. That decides what to look for first.
- **What's the core object?** Find the one or two classes everything else hangs off (`Connection` and `Cursor`, `ZipFile`, `Request` and `Response`).
- **Bytes or text?** Half of all stdlib confusion. Crypto, sockets and compression want `bytes`; you `.encode()` on the way in and `.decode()` on the way out (Ch 9).
- **Who closes it?** Look for `close()`, `__enter__` in `dir()` (it works with `with`), or a `commit()`.
- **What does it raise?** Look for names ending in `Error` in `dir(module)`; that's the Errors row of the card.
- **Is there a one-shot helper?** Many modules have a simple function for the common case (`hmac.digest`, `json.loads`, `subprocess.run`, `urllib.request.urlopen`). Start there.

```
>>> print(*[n for n in dir(sqlite3) if n.endswith("Error")])
DataError DatabaseError Error IntegrityError InterfaceError InternalError NotSupporte…
>>> "__enter__" in dir(sqlite3.Connection)
True
```

IN OTHER LANGUAGES

The method is language-neutral. In Java the REPL is `jshell` and the docs are Javadoc; in Rust, `cargo doc --open` and the docs.rs Examples section; in C, `man 3 qsort`. The questions (shape, core object, lifecycle, errors, traps) don't change.

Practise, then stamp

NOW PRACTISE (practice/ folder, no installs needed)

Run `python -m practice check 7.1` (then the next id). Tests check what your code **does**, not how it looks, so any correct solution passes. After a pass, `python -m practice alt 7.1` shows other valid solutions and their trade-offs.

`7.1` learn an unseen module Map a module's public surface
`7.2` learn an unseen module Read a signature with inspect
`7.3` learn an unseen module Did you mean...? (difflib)
`7.4` learn an unseen module Zip and unzip in memory (zipfile)
`7.5` learn an unseen module Wrap text for a terminal (textwrap)
`7.6` mini-project Library card generator

PASSPORT STAMP ask the same question of every new language (Appendix B)

9. Modules and build

A module is a `.py` file (or compiled C `.so`) found on `sys.path`; `module.__file__` says which. Packages are folders of modules.

12. Stdlib and tooling

Built-in introspection: `dir`, `help`, `type`, `inspect.signature`, `inspect.getsource`, `python -m pydoc`. Most of the stdlib is readable Python source.

DEEP DIVE (your vault)

Vault notes: 11.10 Standard-library method: inspect by contract · 10.2 Import is execution with caching · 10.7 Public APIs and `__all__` · 8.15 Function attributes and introspection · 15.20 Reading CPython source responsibly.